Iteration#05

Riley Ashok, Presthika Vijaykumar

March 2026

1 Develop Detailed Pseudocode

Functionality	**Purpose**	**Input**	**Output**
Drone	To create a new drone object	droneID position velocity	Confirmation message of creation of drone

Table 1: Functionalities of Drone class

Functionality	**Purpose**	**Input**	**Output**
Drone Registration	Add drone to fleet and initialize to system	droneID position velocity	Confirmation or Error Message

Table 2: Functionalities of Drone Registration class

Functionality	**Purpose**	**Input**	**Output**
Fleet Manager	To create a fleet of drones to go through a show	DroneID array FleetID	Confirmation message for fleet creation

Table 3: Functionalities of Fleet Manager class

Functionality	Purpose	Input	Output
Mission Planner	To set and plan a mission for the drones to follow, likely a series of shapes	MissionID Shape Selected Drones	Confirmation of created mission

Table 4: Functionalities of Mission Planner class

Functionality	Purpose	Input	Output
SimulationEngine	To visualize the drone show and make a UI	MissionID FleetID	Simulation of the drone show thus far

Table 5: Functionalities of SimulationEngine class

2 Class Interactions and Responsibilities

Class Name	Responsibility	Interacts with	Reason
Drone	Create a drone object to be used in the show	DroneRegistration FleetManager MissionPlanner SimulationEngine	This is the base drone class that all other classes will use for the mission
DroneRegistration	Register a drone	Drone	Takes the created drone and registers it
FleetManager	Manage an array of drone and work with separate fleets of drones at a time	DroneRegistration Drone	Uses DroneRegistration to add drones to the fleet and manage an array of drone objects
MissionPlanner	Add elements to the drone show and direct fleets of drones into different shapes	FleetManager Drone	It uses fleets and drones to create the overall mission and directs the fleets into inputed positions
SimulationEngine	Display the total mission and show to the user and handle the UI	FleetManager MissionPlanner Drone	Shows the fleets and shows a visualization of the overall mission. Can also access individual drones to see their path

Table 6: Interactions between classes

3 Pseudocode

3.1 Drone Class

```
Class Drone
    Private:
        droneID → integer
        position[3] → array
        velocity[3] → array
        acceleration[3] → array
        color → string
    Public:
```

Function Constructor(id, x, y, z, vx, vy, vz, ax, ay, az)
    droneID ← id
    position ← [x, y, z]
    velocity ← [vx, vy, vz]
    acceleration ← [ax, ay, az]
End Function

Function DefaultConstructor()
    droneID ← 0
    position ← $[0, 0, 0]$
    velocity ← $[0, 0, 0]$
    acceleration ← $[0, 0, 0]$
End Function

Function setDroneID(id)
    droneID ← id
End Function
Function setPosition(x, y, z)
    position ← $[x, y, z]$
End Function
Function setVelocity(vx, vy, vz)
    velocity ← $[vx, vy, vz]$
End Function
Function setAcceleration(ax, ay, az)
    acceleration ← $[ax, ay, az]$
End Function
Function setColor(c)
    color ← c
End Function

Function getDroneID() → integer
    return droneID
End Function
Function getPosition()
    return position
End Function
Function getVelocity()
    return velocity
End Function
Function getAcceleration()
    return acceleration
End Function
Function getColor()
    return color
End Function

Function getSpeed()

```
    return magnitude of velocity
End Function
Function getVelocityAngle()
    return velocity angle in degrees
End Function
Function getDistanceFromOrigin()
    return distance from origin
End Function
Function getDistanceFromOther(otherDrone)
    Calculate dx, dy, dz
    return distance
End Function
Function getAccelerationMagnitude()
    return magnitude of acceleration
End Function

Function displayInfo()
    Output droneID
    Output position
    Output velocity
    Output acceleration
    Output color
End Function

Function updatePosition(time)
    position ← position + velocity × time
    Output updated position
End Function
Function updateVelocity(ax, ay, az, time)
    velocity ← velocity + acceleration × time
    Output updated velocity
End Function
Function applyAcceleration(ax, ay, az, time)
    Call updateVelocity(ax, ay, az, time)
    Call updatePosition(time)
End Function

Function stop()
    velocity ← [0, 0, 0]
    Output "Drone stopped"
End Function
Function reset()
    position ← [0, 0, 0]
    velocity ← [0, 0, 0]
    acceleration ← [0, 0, 0]
    Output "Drone reset"
```

```
95:         End Function

96:         Function detectCollision(otherDrone, collisionDistance)
97:             Calculate distance
98:
99:  if distance < collisionDistance then
100:                    Output "Collision detected"
101:                    return 1
102:
103: else
104:                    Output "No collision"
105:                    return 0
106:
107: end if
108:        End Function
109:        Function avoidCollision(otherDrone, collisionDistance)
110:
111: if detectCollision(otherDrone, collisionDistance) == 1 then
112:                    velocity ← negative of velocity
113:                    Output updated velocity
114:
115: end if
116:        End Function
117: End Class
```

3.2 FleetManager Class

```
1:  Class FleetManager
2:      Private:
3:          drone[] → array of Drone objects
4:          droneCount → integer
5:      Public:
6:          Function AddDrone(droneID)
7:
8:  for each drone in drone[] do
9:
10:     if drone.ID == droneID then
11:                         Output "Drone with ID [droneID] already exists"
12:                         return
13:
14:     end if
15:
16: end for
17:             drone[droneCount].ID ← droneID
18:             droneCount ← droneCount + 1
19:             Output "Drone with ID [droneID] added"
```

```
        End Function

        Function DeleteDrone(droneID)

for each drone in drone[] do

    if drone.ID == droneID then
                        Shift all drones after current index left by one
                        droneCount ← droneCount - 1
                        Output "Drone with ID [droneID] deleted"
                        return

    end if

end for
            Output "Drone with ID [droneID] not found"
        End Function
End Class
```

3.3 MissionPlanner Class

```
Class MissionPlanner
    Private:
        missionID → integer
        missionName → string
        missionType → string
        missionRunNames[10] → array
        missionRunCount → integer
    Public:
        Function Constructor(ID, name, type)
            missionID ← ID
            missionName ← name
            missionType ← type
            missionRunCount ← 0
        End Function

        Function DefaultConstructor()
            missionID ← 0
            missionName ← "Default Mission"
            missionType ← "Default Type"
            missionRunCount ← 0
        End Function

        Function setMissionID(ID)
            missionID ← ID
        End Function
```

```
        Function setMissionName(name)
            missionName ← name
        End Function
        Function setMissionType(type)
            missionType ← type
        End Function

        Function getMissionID() → integer
            return missionID
        End Function
        Function getMissionName() → string
            return missionName
        End Function
        Function getMissionType() → string
            return missionType
        End Function

        Function displayMissionInfo()
            Output "Mission Info"
            Output missionID
            Output missionName
            Output missionType
        End Function

        Function assignMission(fleet)
            Assign mission to fleet using fleet ID
        End Function
        Function assignMission(drone)
            Assign mission to drone
        End Function

        Function deleteMission()
            missionID ← 0
            missionName ← "Deleted Mission"
            missionType ← "Deleted Type"
        End Function

        Function addMissionElement(name)

if missionRunCount < 10 then
                    missionRunNames[missionRunCount] ← name
                    missionRunCount ← missionRunCount + 1

else
                    Output "Cannot add more elements"

end if
```

```
66:         End Function
67: End Class
```

3.4 SimulationsEngine class

```
1: Class SimulationEngine
2:     Public:
3:         Function RunStep(timeStep)
4:
5: if timeStep ≤ 0 then
6:                     Output "Invalid time step"
7:                     return
8:
9: end if
10:             Output "Running simulation step with time step: [timeStep] seconds"
11:         End Function

12:         Function DisplayStatus()
13:             Output "Displaying simulation status..."
14:         End Function
15: End Class
```

Full Code can be seen on the github: https://github.com/rileyashok/DroneSims